

Pontificia Universidad Católica Madre y Maestra.

Campus Santiago

Facultad de Ciencias e Ingeniería

Escuela de Ingeniería en Computación y Telecomunicaciones



Clave del curso: ITT363-171 / 5210

Nombre de la materia: Proyecto Integrador ITT

Nombres de los integrantes: **Matricula**

Daury Vásquez 1015-5870

Marcos López 1015-4910

Asignación:

Simulador de lecturas de estación meteorológica

Maestro (A):

Carlos Alfredo Camacho

Fecha De Entrega:

22 de mayo de 2026

Índice

Introducción	3
Objetivos	4
Objetivo General	4
Objetivos Específicos.....	4
Marco Teórico.....	5
Internet de las Cosas (IoT).....	5
Protocolo MQTT.....	5
Componentes principales de MQTT	5
Publicador (Publisher)	5
Suscriptor (Subscriber)	5
Broker	5
Calidad de Servicio (QoS)	6
Estaciones Meteorológicas.....	6
Eclipse Paho MQTT	7
Github como Sistema de Control de Versiones	7
Descripción del Problema	8
Diseño de la Solución	9
Arquitectura General.....	9
Diseño del Publicador MQTT.....	9
Diseño del Suscriptor MQTT.....	9
Implementación del Publicador MQTT	10
Implementación del Suscriptor	11
Explicación del Código.....	11
Clase SimuladorEstacion	11
Clase LectorEstacion	12
Pruebas Realizadas.....	12
Prueba de Conexión al Broker	12
Prueba de Publicación de Datos.....	13
Prueba de Recepción de Mensajes.....	13
Prueba de Simulación de Múltiples Estaciones	14
Repositorio del Proyecto.....	15
Conclusión	16
Bibliografía	17

Introducción

El Internet de las Cosas (IoT) ha permitido la interconexión de dispositivos capaces de recopilar, transmitir y procesar información en tiempo real. Dentro de este ecosistema, las estaciones meteorológicas inteligentes representan una aplicación importante, ya que permiten monitorear variables ambientales como temperatura, humedad, presión atmosférica, velocidad del viento y precipitaciones, facilitando la toma de decisiones en distintos sectores como agricultura, telecomunicaciones, investigación y gestión ambiental.

Para la comunicación entre dispositivos IoT se utilizan protocolos ligeros y eficientes, siendo MQTT (Message Queuing Telemetry Transport) uno de los más utilizados debido a su bajo consumo de ancho de banda, rapidez y facilidad de implementación bajo el modelo Publisher/Subscriber. Este protocolo permite que múltiples dispositivos publiquen información hacia un broker central, mientras otros dispositivos pueden suscribirse a los tópicos de interés para recibir los datos en tiempo real.

El presente proyecto tiene como objetivo desarrollar una aplicación en Java capaz de simular múltiples estaciones meteorológicas y transmitir sus lecturas mediante MQTT utilizando el servidor público proporcionado por la Escuela de Ingeniería de la PUCMM: *mqtt.eict.ce.pucmm.edu.do*. Para ello, se implementó un sistema publicador encargado de generar datos meteorológicos simulados y enviarlos a través de una jerarquía organizada de topics, así como un sistema suscriptor capaz de recibir y visualizar dicha información desde la consola.

La solución desarrollada permite simular más de una estación meteorológica de manera simultánea, manteniendo una estructura escalable y organizada para la transmisión de datos. Además, se utilizaron herramientas como Java, MQTT Explorer, IntelliJ IDEA y la librería Eclipse Paho MQTT para la implementación del sistema, integrando posteriormente el proyecto en un repositorio Github para su distribución y evaluación.

Objetivos

Objetivo General

Desarrollar un sistema de simulación de estaciones meteorológicas utilizando el protocolo MQTT para la transmisión de datos en tiempo real, permitiendo la comunicación entre múltiples estaciones y un cliente suscriptor mediante el uso del broker público de la PUCMM.

Objetivos Específicos

- Implementar una aplicación publicadora en Java capaz de simular lecturas meteorológicas y transmitirlos mediante MQTT.
- Simular múltiples estaciones meteorológicas con diferentes valores de sensores ambientales.
- Diseñar una jerarquía de topics organizada para la publicación de datos utilizando la estructura asignada al grupo.
- Implementar una aplicación suscriptora capaz de recibir y mostrar en consola la información transmitida por las estaciones meteorológicas.
- Utilizar el broker MQTT público `mqtt.eict.ce.pucmm.edu.do` para la comunicación entre publicadores y suscriptores.
- Aplicar mecanismos de reconexión automática y calidad de servicio (QoS) para mejorar la confiabilidad de la transmisión.
- Publicar el proyecto y su documentación en un repositorio GitHub siguiendo la estructura solicitada por el corrector.

Marco Teórico

Internet de las Cosas (IoT)

El Internet de las Cosas, conocido como IoT (Internet of Things), es un concepto tecnológico basado en la interconexión de dispositivos físicos capaces de recopilar, procesar y transmitir información a través de redes de comunicación. Estos dispositivos pueden incluir sensores, actuadores, electrodomésticos inteligentes, sistemas industriales y estaciones meteorológicas, entre otros.

El objetivo principal del IoT es permitir la automatización y el monitoreo en tiempo real de distintos entornos mediante el intercambio constante de datos. En el caso de las estaciones meteorológicas, IoT permite capturar variables ambientales y transmitirlas hacia sistemas centrales para su análisis y visualización en tiempo real.

Protocolo MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo de comunicación ligero diseñado especialmente para sistemas IoT y dispositivos con recursos limitados. Fue desarrollado para facilitar la transmisión eficiente de datos mediante redes con bajo ancho de banda o conexiones inestables.

MQTT funciona bajo el modelo Publisher/Subscriber (Publicador/Suscriptor), donde los dispositivos no se comunican directamente entre sí, sino a través de un intermediario llamado broker.

Componentes principales de MQTT

Publicador (Publisher)

Es el dispositivo o aplicación encargada de enviar información hacia el broker utilizando topics específicos. En este proyecto, el simulador meteorológico actúa como publicador.

Suscriptor (Subscriber)

Es el dispositivo o aplicación que recibe los mensajes publicados en determinados topics. En este proyecto, el lector de estaciones meteorológicas actúa como suscriptor.

Broker

Es el servidor encargado de administrar las conexiones, recibir mensajes de los publicadores y distribuirlos a los suscriptores correspondientes. El broker utilizado en este proyecto es:

mqtt.eict.ce.pucmm.edu.do

Calidad de Servicio (QoS)

MQTT implementa diferentes niveles de calidad de servicio conocidos como QoS (Quality of Service), los cuales determinan la confiabilidad en la entrega de mensajes.

QoS 0 - At most once

El mensaje se envía una sola vez sin confirmación de recepción.

QoS 1 - At least once

El mensaje se entrega al menos una vez y requiere confirmación del receptor. Este fue el nivel utilizado en el proyecto para asegurar la recepción de los datos.

QoS 2 - Exactly once

Garantiza que el mensaje se entregue exactamente una vez, aunque consume más recursos de red.

Estaciones Meteorológicas

Una estación meteorológica es un sistema compuesto por sensores capaces de medir variables ambientales y climáticas. Estas estaciones son utilizadas para monitoreo atmosférico, investigación científica, agricultura y prevención de riesgos climáticos.

En este proyecto se simularon los siguientes sensores:

Sensor	Función
Temperatura	Mide el nivel de calor ambiental
Humedad del aire	Mide la cantidad de vapor de agua presente en el ambiente
Presión atmosférica	Mide la presión ejercida por la atmósfera
Velocidad del viento	Indica la rapidez del viento
Dirección del viento	Determina hacia dónde se dirige el viento
Lluvia acumulada	Registra la cantidad de precipitación
Humedad del suelo	Mide el nivel de humedad presente en el suelo

La simulación de estos sensores se realizó mediante generación dinámica de datos aleatorios controlados, permitiendo representar variaciones climáticas similares a las de un entorno real.

Eclipse Paho MQTT

Eclipse Paho es una librería de código abierto desarrollada para facilitar la implementación del protocolo MQTT en diferentes lenguajes de programación, incluyendo Java.

Esta librería proporciona herramientas para:

- Conectarse a un broker MQTT.
- Publicar mensajes.
- Suscribirse a topics.
- Manejar reconexiones automáticas.
- Configurar niveles QoS.

En este proyecto se utilizó Eclipse Paho para implementar tanto el publicador como el suscriptor en Java.

Github como Sistema de Control de Versiones

Github es una plataforma basada en Git utilizada para almacenar, administrar y compartir proyectos de software. Permite mantener un historial de cambios, colaborar entre múltiples desarrolladores y facilitar la distribución del código fuente.

En este proyecto, Github fue utilizado para:

- Publicar el código fuente.
- Compartir la documentación.
- Organizar las evidencias del proyecto.
- Facilitar la revisión por parte del profesor o corrector.

Descripción del Problema

En los sistemas modernos de monitoreo ambiental es necesario recopilar y transmitir información meteorológica en tiempo real de manera eficiente, organizada y escalable. Las estaciones meteorológicas generan constantemente datos relacionados con variables climáticas como temperatura, humedad, presión atmosférica, velocidad del viento y precipitaciones, los cuales deben ser enviados a sistemas capaces de procesarlos y visualizarlos adecuadamente.

Uno de los principales desafíos en este tipo de soluciones es garantizar una comunicación ligera y eficiente entre múltiples dispositivos, especialmente cuando se trabaja en entornos IoT donde pueden existir limitaciones de ancho de banda y una gran cantidad de dispositivos conectados simultáneamente. Además, resulta importante mantener una estructura organizada para la transmisión de datos, permitiendo identificar fácilmente la estación y el sensor que genera cada lectura.

A partir de esta necesidad, se plantea el desarrollo de una aplicación capaz de simular múltiples estaciones meteorológicas y transmitir sus lecturas utilizando el protocolo MQTT mediante un broker centralizado. El sistema debe permitir la publicación continua de datos meteorológicos utilizando una jerarquía de topics organizada y escalable, de forma que múltiples estaciones puedan coexistir dentro de la misma infraestructura de comunicación.

Asimismo, se requiere implementar un sistema suscriptor que pueda recibir toda la información transmitida y mostrarla en tiempo real a través de la consola, permitiendo monitorear el comportamiento de cada estación meteorológica simulada.

Además del objetivo funcional del sistema, uno de los principales retos del proyecto consiste en lograr la integración futura entre el hardware físico de una estación meteorológica y el software encargado de procesar y transmitir la información. Debido a las limitaciones de tiempo y a la complejidad de integrar sensores reales en esta etapa de la asignatura Proyecto Integrador, se optó por desarrollar inicialmente una simulación del comportamiento de los sensores meteorológicos. Esta estrategia permite avanzar en el diseño de la arquitectura de comunicación, la lógica de transmisión de datos y el manejo del protocolo MQTT, facilitando posteriormente la migración hacia una implementación física real sin afectar la estructura general del sistema.

Para cumplir con estos requerimientos, se utilizó el broker público proporcionado por la PUCMM `mqtt.eict.ce.pucmm.edu.do`, junto con aplicaciones desarrolladas en Java utilizando la librería Eclipse Paho MQTT, logrando así una solución funcional orientada a sistemas IoT y comunicación en tiempo real.

Diseño de la Solución

Arquitectura General

Para dar solución al problema planteado, se diseñó una arquitectura basada en el modelo de comunicación Publisher/Subscriber del protocolo MQTT. La solución desarrollada permite simular múltiples estaciones meteorológicas capaces de generar datos ambientales y transmitirlos en tiempo real hacia un broker MQTT centralizado, mientras un sistema suscriptor recibe y visualiza la información desde la consola.

La arquitectura fue diseñada pensando en la escalabilidad y en la futura integración con hardware físico real. Por esta razón, se utilizó una estructura modular donde las estaciones meteorológicas funcionan de manera independiente y publican sus datos utilizando una jerarquía organizada de topics.

Diseño del Publicador MQTT

El publicador fue desarrollado en Java utilizando la librería Eclipse Paho MQTT. Su función principal es simular el comportamiento de múltiples estaciones meteorológicas y transmitir periódicamente las lecturas generadas.

Cada estación mantiene un conjunto independiente de variables climáticas, permitiendo representar diferentes condiciones ambientales.

Diseño del Suscriptor MQTT

El suscriptor fue diseñado para conectarse al broker MQTT y escuchar todos los mensajes pertenecientes al grupo mediante el topic global:

/itt363-grupo2/#

Gracias al uso del wildcard #, el sistema puede recibir automáticamente todos los datos publicados por cualquier estación meteorológica perteneciente al grupo 2.

Cuando se recibe un mensaje, el suscriptor:

1. Identifica la estación de origen.
2. Detecta el tipo de sensor.
3. Extrae el valor recibido.
4. Asigna automáticamente la unidad correspondiente.
5. Muestra la información organizada en consola.

Esto permite realizar monitoreo en tiempo real de todas las estaciones simuladas

Estructura Jerárquica de Topics

Para organizar la información transmitida se utilizó una jerarquía de topics estructurada de la siguiente manera:

/itt363-grupo2/estacion-X/sensores/tipo_sensor

Implementación del Publicador MQTT

La aplicación publicadora fue desarrollada en Java utilizando la librería Eclipse Paho MQTT. Su función principal es simular múltiples estaciones meteorológicas y transmitir periódicamente las lecturas generadas hacia el broker MQTT proporcionado por la PUCMM. Además de las aplicaciones desarrolladas en Java, se utilizó la herramienta MQTT Explorer para monitorear visualmente los topics y verificar en tiempo real la correcta publicación de los mensajes enviados por las estaciones meteorológicas simuladas.

La conexión se realiza mediante el protocolo TCP utilizando el puerto 1883, correspondiente al puerto estándar de MQTT.

En el código, la dirección del broker fue definida de la siguiente manera:

```
private static final String BROKER =  
"tcp://mqtt.eict.ce.pucmm.edu.do:1883";
```

Cada cliente MQTT utiliza un identificador único (clientId) generado dinámicamente para evitar conflictos entre conexiones simultáneas.

El broker requiere autenticación mediante usuario y contraseña asignados a cada grupo. En este proyecto se utilizaron las credenciales correspondientes al grupo 2:

```
private static final String USUARIO = "itt363-grupo2";  
private static final String CLAVE = "knDH2P6N4w9g";
```

Las credenciales son configuradas mediante el objeto `MqttConnectOptions`, el cual también habilita la reconexión automática en caso de pérdida de conexión.

```
MqttConnectOptions opciones = new MqttConnectOptions();  
opciones.setUsername(USUARIO);  
opciones.setPassword(CLAVE.toCharArray());  
opciones.setAutomaticReconnect(true);
```

Generación de Datos Aleatorio

El sistema simula el comportamiento de diferentes sensores meteorológicos utilizando valores aleatorios controlados. Cada estación mantiene variables independientes para representar condiciones climáticas distintas.

Para lograr una simulación más realista, se implementó un mecanismo de variación gradual conocido como Random Walk, donde los valores aumentan o disminuyen ligeramente en cada actualización en lugar de cambiar bruscamente.

Implementación del Suscriptor

La aplicación suscriptora fue desarrollada en Java utilizando la librería Eclipse Paho MQTT y tiene como función recibir y mostrar en tiempo real los datos enviados por las estaciones meteorológicas simuladas.

El suscriptor se conecta al broker MQTT utilizando las credenciales del grupo 2 y se suscribe al topic global:

```
/itt363-grupo2/#
```

Gracias al wildcard #, la aplicación puede recibir automáticamente todos los mensajes publicados por cualquier estación perteneciente al grupo.

Cuando un mensaje es recibido, el sistema identifica la estación de origen, el sensor correspondiente y el valor transmitido, mostrando la información organizada en la consola junto con su unidad de medida.

Además, se implementó un callback MQTT para manejar la recepción de mensajes y detectar pérdidas de conexión, permitiendo la reconexión automática al broker.

Durante las pruebas también se utilizó MQTT Explorer para verificar visualmente la recepción de mensajes y confirmar el correcto funcionamiento de la comunicación MQTT entre publicador y suscriptor.

Explicación del Código

Clase SimuladorEstacion

La clase SimuladorEstacion es la encargada de generar y publicar los datos meteorológicos hacia el broker MQTT.

Variables Globales

Se definen variables globales para almacenar la dirección del broker, usuario y contraseña utilizados para la conexión MQTT.

Clase EstadoClima

La clase interna EstadoClima almacena las variables meteorológicas de cada estación, como temperatura, humedad, presión, viento y lluvia. Cada estación posee su propio estado climático independiente.

Método actualizar()

Este método modifica los valores de los sensores utilizando variaciones aleatorias controladas para simular cambios climáticos graduales y realistas.

Método publicar()

El método publicar() se encarga de enviar los datos hacia el broker MQTT utilizando los topics definidos para cada sensor y estación meteorológica.

Clase LectorEstacion

La clase LectorEstacion funciona como suscriptor MQTT y permite visualizar en tiempo real los datos recibidos desde las estaciones meteorológicas.

Callback MQTT

Se implementó un callback MQTT para manejar eventos como pérdida de conexión y recepción de mensajes.

Método messageArrived()

Este método se ejecuta automáticamente cada vez que llega un nuevo mensaje desde el broker MQTT.

Parsing del Topic

El sistema divide el topic recibido para identificar la estación meteorológica y el sensor correspondiente.

Ejemplo:

/itt363-grupo2/estacion-1/sensores/temperatura

Presentación en Consola

Finalmente, la información recibida es organizada y mostrada en consola indicando la hora, estación, sensor y valor correspondiente con su unidad de medida.

Pruebas Realizadas

Prueba de Conexión al Broker

Se verificó la conexión exitosa al broker público de la PUCMM `mqtt.eict.ce.pucmm.edu.do` utilizando las credenciales asignadas al grupo 2. Tanto el publicador como el suscriptor lograron conectarse correctamente al servidor MQTT.

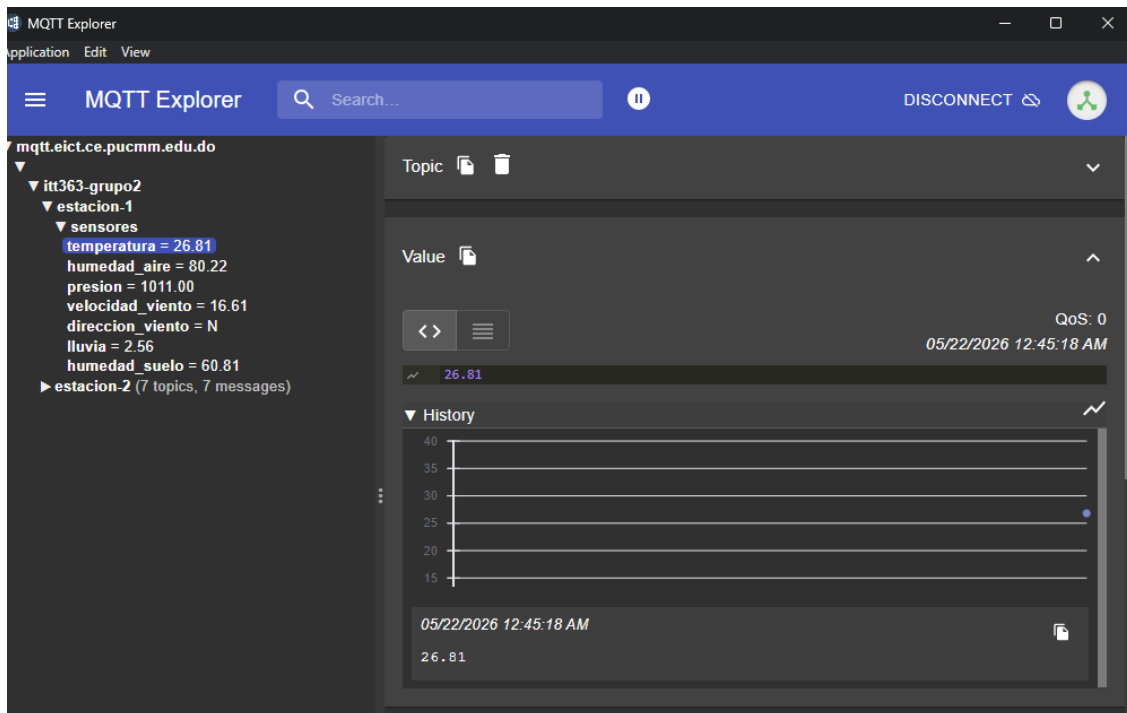
```
C:\Users\daury\.jdk\openjdk-25.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.2\lib\idea_rt.jar=55668" -Dfile.encoding=UTF-8
Publicador conectado exitosamente.
[Estacion 1] -> T: 28,2°C | H: 70,0% | Viento: N | Lluvia Acum: 0,0mm
[Estacion 2] -> T: 22,5°C | H: 70,5% | Viento: N | Lluvia Acum: 0,0mm
--- Sincronización completada ---
```

Prueba de Publicación de Datos

Se realizaron pruebas de publicación de mensajes desde el simulador meteorológico, comprobando que los datos de los sensores fueran enviados correctamente hacia los topics definidos.

Ejemplo de topic probado:

/itt363-grupo2/estacion-1/sensores/temperatura



Prueba de Recepción de Mensajes

La aplicación suscriptora logró recibir correctamente todos los mensajes enviados por las estaciones meteorológicas mediante el topic global:

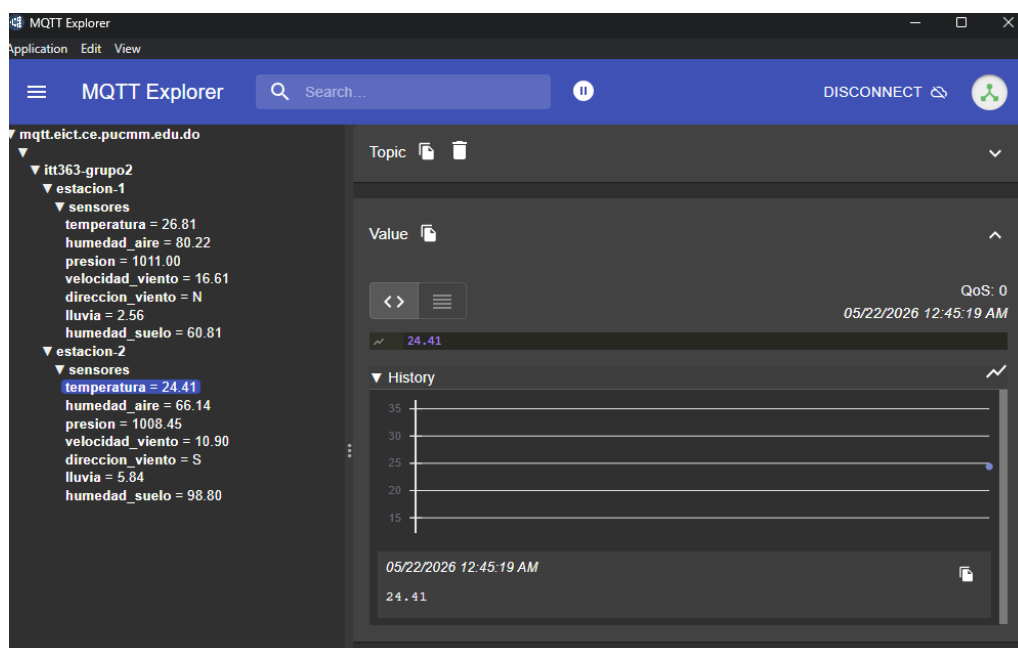
/itt363-grupo2/#

ESTACION METEOROLOGICA - GRUPO 2 - LECTOR			
Broker : tcp://mqtt.eict.ce.pucmm.edu.do:1883			
Topic : /itt363-grupo2/#			
[HORA]	[ESTACION]	[SENSOR]	[VALOR]
[00:41:20]	estacion-1	temperatura	29.43 °C
[00:41:20]	estacion-1	humedad_aire	70.64 %
[00:41:21]	estacion-1	presion	1011.12 hPa
[00:41:21]	estacion-1	velocidad_viento	12.54 km/h
[00:41:21]	estacion-1	direccion_viento	NW
[00:41:21]	estacion-1	lluvia	0.00 mm
[00:41:21]	estacion-1	humedad_suelo	38.40 %
[00:41:21]	estacion-2	temperatura	21.87 °C
[00:41:21]	estacion-2	humedad_aire	69.61 %

La información fue mostrada en consola indicando estación, sensor, valor y unidad de medida correspondiente.

Prueba de Simulación de Múltiples Estaciones

Se verificó el funcionamiento simultáneo de múltiples estaciones meteorológicas, comprobando que cada una publicara datos independientes utilizando diferentes topics dentro de la misma jerarquía MQTT.



Repositorio del Proyecto

<https://github.com/Daury-201/EstacionMetereologica.git>

Conclusión

El desarrollo de esta asignación permitió comprender e implementar los conceptos fundamentales del protocolo MQTT dentro de un entorno orientado al Internet de las Cosas (IoT). A través de la creación de un sistema capaz de simular múltiples estaciones meteorológicas, fue posible establecer una comunicación eficiente en tiempo real entre aplicaciones publicadoras y suscriptoras utilizando el broker MQTT proporcionado por la PUCMM.

La solución desarrollada en Java logró generar y transmitir datos meteorológicos simulados de manera organizada mediante una jerarquía estructurada de topics, permitiendo identificar fácilmente cada estación y sus respectivos sensores. Asimismo, el sistema suscriptor fue capaz de recibir y visualizar correctamente toda la información publicada, demostrando el funcionamiento adecuado del modelo Publisher/Subscriber implementado por MQTT.

Durante la asignación también se adquirieron conocimientos importantes relacionados con la configuración de clientes MQTT, autenticación, uso de QoS, manejo de reconexiones automáticas y monitoreo de mensajes utilizando herramientas como MQTT Explorer. Además, la simulación permitió representar comportamientos climáticos más realistas mediante variaciones dinámicas de los sensores, acercando la práctica a un escenario real de monitoreo ambiental.

Uno de los aspectos más importantes de esta etapa fue preparar la base de comunicación y software necesaria para una futura integración con hardware físico real. La implementación de simulaciones permitió avanzar en el diseño de la arquitectura del sistema sin depender inicialmente de sensores físicos, optimizando el tiempo de desarrollo y facilitando futuras expansiones.

En conclusión, los resultados obtenidos demuestran que MQTT es una tecnología eficiente, ligera y escalable para aplicaciones IoT, permitiendo una transmisión confiable de información en tiempo real. La asignación desarrollada constituye una base sólida para futuras implementaciones más avanzadas relacionadas con monitoreo ambiental, automatización y sistemas inteligentes conectados.

Bibliografía

Banks, A., & Gupta, R. (2014). *MQTT Version 3.1.1*. OASIS Standard. Recuperado de [OASIS MQTT Specification](#)

Eclipse Foundation. (s.f.). *Eclipse Paho Java Client*. Recuperado de [Eclipse Paho MQTT Java Client Documentation](#)

GitHub, Inc. (s.f.). *GitHub Documentation*. Recuperado de [GitHub Official Website](#)

HiveMQ. (s.f.). *What is MQTT and How Does It Work?* Recuperado de [HiveMQ MQTT Essentials](#)

JetBrains. (s.f.). *IntelliJ IDEA Documentation*. Recuperado de [IntelliJ IDEA Official Website](#)

MQTT.org. (s.f.). *MQTT: The Standard for IoT Messaging*. Recuperado de [MQTT Official Website](#)

Oracle Corporation. (s.f.). *Java Documentation*. Recuperado de [Oracle Java Documentation](#)

PUCMM. (2026). *Material de apoyo de la asignatura Proyecto Integrador ITT-363*. Pontificia Universidad Católica Madre y Maestra.

Schmitt, T. (s.f.). *MQTT Explorer*. Recuperado de [MQTT Explorer Official Website](#)